

10. **Why score-based grading instead of letter-grade menu selection** — the design decision to make students enter a raw numeric score (0–100) rather than directly picking a letter grade from a menu, and why this is more faithful to how real grading works (grade is *derived* from a score, not self-declared), even though it can't fully prevent someone from lying about their score.
11. **scoreToGrade and pass-by-reference (&)** — explain the & in string &letterGrade, double &gradePoint: how reference parameters let a void function produce two outputs at once by modifying the caller's original variables directly, instead of returning just one value. Contrast this clearly with normal pass-by-value.
12. **If/else-if chains for score ranges vs switch for discrete menus** — why a chain of if (score >= X) comparisons was needed for continuous numeric ranges, whereas an earlier version used switch for a fixed discrete 1–10 menu choice, and why switch doesn't work well for ranges.
13. **The -1.0 sentinel value / signaling errors without exceptions** — how a function can signal "something went wrong" through a special return value that's outside the range of valid real answers, and how the caller checks for it in a loop, as a simple beginner-friendly alternative to more advanced error-handling mechanisms.
14. **Duplicate-selection prevention in selectCourses** — the nested loop checking a vector<int> chosenIndices against each new choice, and why a manual search loop was used instead of a more advanced tool.
15. **The GPA/CGPA formula itself** — $\Sigma(\text{creditHours} \times \text{gradePoint}) / \Sigma(\text{creditHours})$, walked through with a tiny worked example (2–3 fake courses with credit hours and grades), showing exactly how the accumulator variables totalQualityPoints and totalCreditHours build up across the loop in calculateCGPA. Include the division-by-zero guard and why it's there.
16. **Ternary operator** — (uniChoice == 1) ? "... " : "... " in displaySummary, what it means, and why it's only appropriate for exactly two outcomes.
17. **Output formatting: fixed and setprecision(2)** — why raw doubles print ugly/inconsistent values, and how these two manipulators from <iomanip> fix that, and that the setting persists on cout for the rest of the program once applied.
18. **Signed vs unsigned comparison warning** — briefly explain why comparing an int against .size() (which returns an unsigned type) can trigger a compiler warning, even though it works fine here in practice, and how a static_cast<int>() would silence it.
19. **Student ID checks are validation, not authentication** — an honest caveat slide: this format-checking doesn't verify someone is really enrolled anywhere, it only checks the string *looks* like a real ID of that university's invented format — useful to state clearly if a lecturer probes on security/authenticity.
20. **Design/scope decisions and possible extensions** — briefly mention what was deliberately left out or simplified (e.g. non-numeric grade categories like X/Z/IC/AD, no real persistence/file storage, no real authentication) and what could be added later (e.g. saving results to a file, supporting more universities, semester-by-semester tracking across a full CGPA rather than one semester's GPA).

Also include a short "live code walkthrough" slide near the end that shows the full main() function with each line annotated (via hover or click) tracing exactly which function produced which variable and where it flows next — this should visually reinforce slide 9's data-flow explanation using the *actual* final code.

Make sure code snippets shown match idiomatic beginner C++ (level 100 university course, first time using vectors/structs/references) — don't rewrite the code to be more advanced than what's already there; the goal is explaining the actual code, not improving it.